Probabilistic Cascading Failure and Agent Assisted Root Cause Analysis for Microservice Systems

Anish C, Dhanush K Murthy, K Bharath Parmar, Royson Salis

***Abstract*—When a microservice fails the failure does not stay where it starts: a slow datastore drags down the services that query it, their callers time out, retry layers amplify the load, and within a minute an operator faces dozens of simultaneous alarms in which the originating fault cannot be told from its victims by severity alone. Recent diagnosis methods localise the fault with graph neural networks over fused metrics, traces and logs, which needs labelled failures, GPU inference and an output the on-call engineer cannot audit. We build the alternative as a complete, unsupervised, single-core pipeline that observes containers from outside the application and assists the operator with a cause, a prediction of what fails next, a fault type and a runbook. Its core is a probabilistic cascading-failure model whose per-edge propagation probabilities are learned online as Beta–Binomial posteriors; a hybrid detector that lets a per-service Isolation Forest and a robust rule layer jointly raise the alarm, with container facts (stopped, crash-looping) treated as evidence rather than degree; a ranker that separates a service's own deviation from what its neighbours explain; and call directions learned from telemetry rather than from service names. We evaluate on 217 minutes of real telemetry from two containerised systems with 62 faults injected blind to the detector, and on 178 held-out synthetic faults over 16 topologies, with bootstrap intervals, paired tests, an ablation and a controlled amplification study. The hybrid names the true cause first for 74% of real faults against 65% for the rule layer (paired difference +9.7 points, 95% CI [+1.6, +17.7]); the learned cascade model discriminates better than fixed edge probabilities (AUROC 0.847 against 0.784) on 80 observed cascades; learned directions stay correct when service names carry no information (28/31 against 0/31); and change-point alarms roughly halve the median detection delay on the real systems (10.5 s to 4.9 s). We also report what did not work, and a measurement bug that had hidden half of the real faults. The pipeline costs about 6 to 7 ms per five-second window on the systems tested.**

***Index Terms*—AIOps, cascading failures, root cause analysis, probabilistic graphical models, anomaly detection, calibration, change-point detection, microservices.**

I. Introduction

MICROSERVICE architectures decompose an application into many small services that communicate over the network [1]. The style buys independent deployment and horizontal scaling, and it introduces a failure mode monolithic systems do not have: a fault does not stay where it starts. When a database slows, the service querying it slows while waiting; the services calling that service time out; a retry layer amplifies the load; and within a minute a dozen services are alarming at once. Practitioners call the result an alert storm. Its cost is not the alerts themselves but the time an engineer spends deciding which one to act on [2], [3].

Two properties of this phenomenon make it tractable. First, failures propagate along the call graph, from a callee to the callers that wait on it, and not arbitrarily. Second, propagation is *probabilistic*: a caller protected by a timeout, a cache or a circuit breaker may survive a failing dependency while an unprotected one will not. A system that knows the call graph and how likely propagation is along each edge can answer two questions that a ranked alert list cannot: which service is the origin, and which services not yet affected are about to be pulled in.

Recent work answers the first question with heavy machinery. Multimodal methods fuse metrics, traces and logs into learned representations over temporal knowledge graphs and localise with graph convolution or autoencoders [4]–[6]. They are effective, and they carry three costs that matter in practice: they need labelled or curated failure data that most organisations do not have; they need GPU inference, which pushes them out of the real-time path; and their output, a cluster identifier or an embedding ranking, cannot be checked by the engineer against the underlying measurement.

This paper builds the alternative end to end and measures it carefully. The system is an *agent-assisted* pipeline in the following concrete sense: a collector agent observes the containers from outside the application, and a diagnosis assistant turns the observations into a cause, a forecast of what fails next, a fault type, a runbook and an explanation the operator can verify. No language model is involved; the assistant is a transparent composition of statistical models, and we discuss where a learned or language-model agent could be added in Section XI.

A. Contributions

1) **A probabilistic cascading-failure model with learned propagation.** Per-edge propagation probabilities are Beta–Binomial posteriors updated from observed incidents; risk flows over the call graph by noisy-OR (the independent-cascade model). On 80 observed cascades it discriminates better than fixed edge probabilities (AUROC 0.847 vs 0.784) and is no worse calibrated (Section VII).
2) **A hybrid detector with an honest operating contract.** A per-service Isolation Forest and a robust rule layer jointly decide the alarm, and container facts such as a stopped or crash-looping service count as evidence rather than degree. We show that false-alarm *episodes*

The authors are with the Department of Artificial Intelligence and Machine Learning, Mangalore Institute of Technology and Engineering, Mangalore, India.

alone can hide an engine that is in alarm most of the time, and report alarm time as a second contract (Section VI).

3) **Fact-aware, decomposition-based root-cause ranking.** Each service's deviation is split into the part its neighbours explain and the part they do not, and a personalised random walk collects root-cause mass at the service whose trouble nothing downstream accounts for (Section VIII).
4) **Call directions learned from telemetry.** A weighted vote of four dynamic and two structural signals recovers call directions with no service-name knowledge (Section V).
5) **A rigorous, reproducible evaluation including negative results.** A blind fault injector provides ground truth independent of the detector; a synthetic generator extends the sample to 16 topologies; every comparison carries bootstrap intervals; and we report an ablation, a controlled amplification study, three mechanisms that did not help, and a ledger-parsing bug that had hidden 14 of 31 faults per system for every engine (Sections IX and X).

II. Related Work

Anomaly detection. Isolation Forest [7], [8] isolates an outlier with few random cuts and needs no density estimate, which suits per-service streams of mixed, skewed metrics. Distance and density methods [9], [10], the histogram score HBOS [11] and the multivariate control chart of Hotelling [12], [13] with shrinkage covariance [14] are natural alternatives and we compare all of them. Robust location and scale from the median and MAD [15], [16] underpin our rule layer. Surveys of streaming and deep anomaly detection [17], [18] and critiques of evaluation practice [19]–[21] motivate our event-level protocol over point-wise F1 with point adjustment.

Root-cause localisation. MonitorRank [22] introduced the personalised random walk over the call graph that we adopt; MicroRCA [23], CloudRanger [24], Microscope [25], AutoMAP [26] and MicroRank [27] extend it with anomaly-weighted transitions and trace spectra. Causal approaches recover structure with the PC algorithm or hierarchical discovery [28]–[30], and learned methods localise with graph autoencoders and multi-source fusion [4]–[6], [31]. These need traces or training; ours needs neither.

Cascade and change. Influence spread under the independent-cascade model [32] is the basis of our propagation prediction, with edge probabilities estimated online. Probabilistic forecasts are assessed with proper scoring rules [33], reliability diagrams and expected calibration error [34], [35]. Detection latency is attacked with classical change-point detection [36], [37]. PageRank [38] and reciprocal-rank fusion [39] inform ranking; Granger causality [40] and transfer entropy [41] inform the direction study; random forests and CART [42], [43] back the fault-type classifier. Chaos-engineering practice [44] and open benchmark applications [45], [46] frame our fault-injection methodology.

III. System Overview and Problem Statement

A system is a set S of N services with a directed call graph: an edge $u \to v$ means u calls v, so a failure travels *against* the arrow, from v to its callers. Every $\Delta = 5\,\mathrm{s}$ the collector produces for each service s a reading $x_s(t) \in (\mathbb{R} \cup \{\varnothing\})^8$: CPU, memory, throttling, network in and out, request rate, error share and p95 latency, with $\varnothing$ for a metric not reported (never zero: a stopped container must be distinguishable from an idle one) together with a container state. At time t the pipeline must decide (i) whether an incident is in progress, (ii) which service is its root cause r, (iii) which currently healthy services will be affected before the incident ends, and (iv) what kind of fault it is. All decisions use only readings up to t.

The collector reads container statistics and the established TCP sockets inside each container over SSH, so no code in the monitored system is changed. This is the property that makes the pipeline deployable on systems that nobody will instrument; the cost is that the observed signals are coarse, which Section X quantifies honestly.

IV. The Evidence-Weighted Graph and Learned Directions

Evidence confidence. An edge's confidence $c(e)$ is the strongest evidence behind it: 1.0 for a call seen in request logs or oriented by a listening port, 0.85 when oriented only by the ephemeral-port range, 0.70 when observed without direction evidence, 0.50 when the direction is unresolved, and 0.40 when declared in the deployment configuration but never seen carrying traffic. The *dependency share* $d(e)$ of an edge out of a caller is its slow-baseline traffic normalised over the caller's edges (a link goes quiet exactly when its callee dies, so the instantaneous rate would understate a dead dependency). The weight is $w(e) = \mathrm{clip}(c(e)\, d(e)\, \phi, 0, 1)$ with ϕ discounting fire-and-forget telemetry edges. The live system passes $w(e)$ into the ranker and the cascade model. *The offline benchmarks of this paper replay recorded observations that do not retain edge evidence, so they use unit weights; the effect of the evidence weighting is therefore not measured here* (Section XI).

V. Learned Call Directions

The collector sees most pairs from both ends (in the recorded runs 15 of 32 pairs in one system and 18 of 36 in the other), so call direction must be recovered before a walk can follow it. The shipped heuristic reads service names: an entry-point name calls a service, a service calls a datastore. It fails on systems whose names carry no role. We instead learn the direction of each observed pair online from the services' own histories, with no labels, no training phase and no service names. Twelve candidate signals were implemented (Fig. 12b); each gives a score $s_m(a, b) \in [-1, 1]$ for "a calls b". The four *dynamic* signals use what the services do during trouble: *upward symptoms* (errors and slow calls of a following resource trouble or a fact at b), *downward traffic drops* (the request rate of b falling when a is in trouble), *implication* ($P(T_a \,|\, T_b) - P(T_b \,|\, T_a)$, where T is "in trouble"),

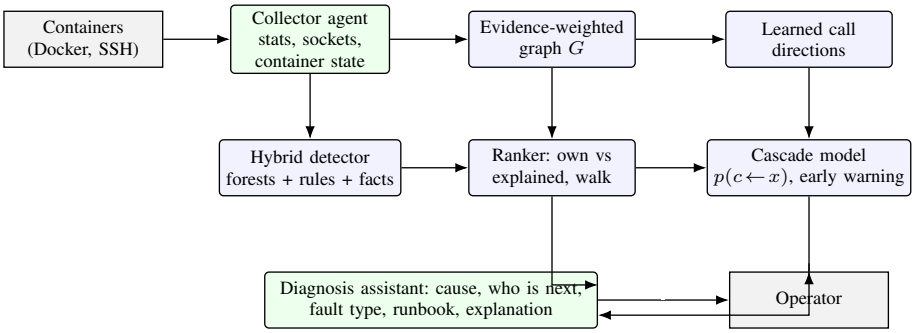


Fig. 1. System overview. The collector agent observes the containers from outside the application. The evidence-weighted graph feeds the learned directions, the hybrid detector, the ranker and the cascade model; the diagnosis assistant turns their outputs into a cause, a forecast of what fails next, a fault type, a runbook and an explanation for the operator.

and a Granger-style F-test on the symptom series. The two *structural* signals use the observed graph: the breadth-first depth of each service from the entry point (the node that talks to something the collector does not watch), and whether a service reports request metrics of its own (a datastore does not, and is therefore a callee). The learned direction is the sign of $\sum_m \omega_m s_m(a,b)$, with weights $\omega_m \in \{0.5, 1, 2\}$ chosen by grid search on the tuning split, and a pair is left undecided if the magnitude is below 0.05. A pair seen from one end only keeps the collector's view. The rule does not read names, but two of its six signals are structural; a time-series-only preset (the four dynamic signals) gives the same ranking accuracy on the real systems (71%).

VI. Hybrid Detection

A service is judged against its own recent past, never against a global threshold, because services differ by orders of magnitude in every metric.

Per-service Isolation Forests. The eight features are split into two groups, each with its own forest of 100 trees trained on a sliding window of the last W=120 readings judged normal: *own* (CPU, memory, throttling), which a dependency cannot cause, and *symptom* (traffic, errors, latency, network), which can be inherited. A reading is scored by the forest built *before* it arrived. With $\mathbb{E}[h(x)]$ the mean path length and $c(\psi)$ the usual normaliser, the raw score $s(x) = 2^{-\mathbb{E}[h(x)]/c(\psi)}$ becomes a level

$$\text{level}(x) = \frac{s(x) - \tilde{s}}{\ell - \tilde{s}}, \qquad \ell = \tilde{s} + \kappa \cdot 1.4826 \cdot \text{MAD}(s_{\text{train}}), \tag{1}$$

where $\tilde{s}$ is the median training score; ℓ is robust to the random trees (κ=3). Only readings with level below one join the training window, so a fault never becomes the baseline. Four refinements were each found by testing and are ablated in Section X: training jitter at each feature's floor (a feature that never moved can still register a later jump); a *range guard* that counts floors beyond the training range (a forest scores an out-of-range value no higher than its most extreme training point); *one-sided features*, so that less CPU, memory, throttling, error or latency than usual is never a fault of the service itself (it happens after every restart); and *two-speed drift*, where a small steady shift (level < 3) is accepted as the new normal within two minutes and a large one is quarantined for thirty.

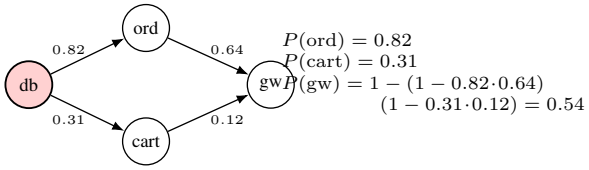


Fig. 2. The cascade model on a four-service example. Edge labels are learned propagation probabilities $p(c \leftarrow x)$; risk at the gateway combines its two routes by noisy-OR.

Rule layer and facts. A robust one-sided z-score per metric (median and MAD, bad direction only) gives a rule score ρ_s normalised so that 1 is its three-sigma line. A container that is exited, paused, restarting or network-detached is a *fact*: evidence, not degree. Two facts that the raw container state misses were added after inspecting the real runs: a `docker stop` exits with code zero, which reads like a finished job, so a service that had been running and then exits is *stopped*; and a crash loop looks healthy between kills, so two outages within three minutes mark it.

Alarm. Let λ_s be a service's forest level and $f_s \in \{0, 1\}$ its fact flag. The project alarms at window t if

$$\max_s f_s = 1 \;\text{ or }\; \max_s \lambda_s \geq \tau_\lambda \;\text{ or }\; \max_s \rho_s \geq \tau_\rho, \tag{2}$$

with τ_λ=2 and τ_ρ=1.25 (the rules alone are trusted only well past their own line). In the live system a fact raises the alarm at once and the other two conditions must hold for two consecutive windows; the offline evaluation requires two consecutive windows for all three, which is the stricter reading. **Two operating contracts.** An operator can budget false-alarm *episodes* per fault-free hour, but one unbroken alarm is a single episode, so an engine can meet an episode budget while alarming almost continuously. We therefore also report *alarm time*, the fraction of fault-free time in alarm. An earlier version of our own detector had a low episode rate and an alarm time of 86% on the real systems; the refinements above bring it to 5%.

VII. A Probabilistic Cascading-Failure Model

Learned edge propagation. Each call edge $c \to x$ carries an unknown probability $p(c \leftarrow x)$ that trouble in x reaches its caller c. Whenever a service x turns abnormal, every healthy

Algorithm 1 One decision window of the diagnosis assistant

Input: readings $x_s(t)$, container states, graph G with learned directions

Output: alarm, ranked causes, likely-next set, fault type, runbook

1: **facts**: mark each service stopped, paused, restarting, detached or crash-looping
2: **score**: forest levels λ_s (own and symptom), rule scores ρ_s, anomalies a_s
3: **alarm**: apply the hybrid rule; if no alarm, update the propagation counts and stop
4: **rank**: services with a fact first, then by the walk share π_s over G
5: **explain**: report own versus symptom evidence and what neighbours explain
6: **predict**: $P(c)$ by Eq. (4) from the top suspect; early-warning flags
7: **diagnose**: classify the fault type of the top suspect; attach its runbook
8: update $\hat{p}(c \leftarrow x)$ from services that newly turned abnormal

caller c is one trial; it succeeds if c turns abnormal within one minute. With k successes in n trials, a Beta prior of mean π_0 and strength m gives the posterior mean

$$\hat{p}(c \leftarrow x) = \frac{k + m\,\pi_0}{n + m}, \tag{3}$$

so a rarely exercised edge starts at its prior and moves with observation, and the model needs no labels: the trials are the system's own history. In all experiments the prior is uninformative (π_0=0.5, m=2). The live system could instead pass the edge's evidence confidence $c(e)$ as π_0; that variant is not evaluated here.

Risk over the graph. Given a suspected root r, risk flows up the call graph by noisy-OR, the independent-cascade model [32] (Fig. 2):

$$P(r) = 1, \qquad P(c) = 1 - \prod_{x \in C(c)} \big(1 - P(x)\,\hat{p}(c \leftarrow x)\big), \tag{4}$$

with $C(c)$ the callees of c. Services above a risk cut are reported as likely next, and $\sum_c P(c)$ estimates the blast radius. **Calibration** is assessed with the Brier score [33] $\mathrm{BS} = \frac{1}{n}\sum_i (p_i - y_i)^2$, the expected calibration error $\mathrm{ECE} = \sum_b \frac{n_b}{n} |\bar{p}_b - \bar{y}_b|$ over equal-width bins [35], and reliability diagrams.

Early warning. For each rising-only feature a least-squares line is fitted over the last two minutes and projected two minutes ahead through the range guard; a service not yet at the alarm level whose trend will reach it is flagged. It fires on gradual faults (memory leaks, throttling), where there is a slope to extrapolate. Separately, online Page–Hinkley change-point detection [36] on the raw metrics can raise the alarm before the forest line is crossed.

VIII. Agent-Assisted Root-Cause Analysis

The assistant runs Algorithm 1 once per window. A caller's errors are usually inherited, so the ranker separates own evidence from symptoms and lets a broken dependency explain away its callers.

Deviation decomposition. With own level $o(s)$, symptom level $\sigma(s)$ and callees $C(s)$,

$$x(s) = \gamma \max_{c \in C(s)} a(c),$$
$$\mathrm{raw}(s) = \max\big(o(s), \max(0, \sigma(s) - x(s))\big), \tag{5}$$

so a service whose symptoms are fully explained by a sick callee drops out, while one that is abnormal in its own resources does not.

Random walk. We rank with a personalised-PageRank walk in the MonitorRank form [22]. A walker on the directed graph steps to a callee v with weight $w(u \to v)\,a(v)$, back to a caller with weight $\rho_{\mathrm{b}}\,a(u)$, and stays with weight $\max(0, a(u) - \max_c a(c), o(u))$, restarting on the anomalous set with probability α=0.15. The stationary share π_s is the score; facts rank first. The stay term lets the source, whose trouble nothing downstream explains, collect the most mass.

Fault type and runbook. At the alarm, 35 features of the top suspect's telemetry (container-state history, exit code, number of separate outages, memory slope, CPU and throttling change, latency and error ratios against a pre-fault baseline, network drop, number of anomalous callers) go to a random forest [42] that names the fault type (crash, stop, pause, crash-loop, flap, memory, brownout, latency, partition); a short runbook of two to four concrete checks per type is attached. This layer changes neither detection nor ranking.

IX. Experimental Methodology

Systems and traffic. Two containerised applications: *ShopFlow*, a 13-service retail checkout system (HTTP fan-out, two Postgres instances, two Redis caches, a NATS consumer), and *LedgerLine*, a 12-service retail-banking transfer system (a saga orchestrator, a double-entry ledger, a third-party FX provider, a batch worker). Both run for 217 minutes under open-loop traffic with a daily wave, bursts and flash crowds.

Blind fault injection. Faults arrive by a non-homogeneous Poisson process whose rate follows the load, each with a class, a role-weighted target, a heavy-tailed duration and a severity; with some probability a follower is spawned in the target's one-hop neighbourhood, so incidents overlap. Faults are physical (`SIGKILL`, pause, cgroup CPU and memory limits, network detach) and nothing is labelled inside the system under test. The injector writes a private ledger with, per fault, the true root cause and the services structurally able to be affected, read only after the run. The two systems contribute 62 faults over seven classes.

Synthetic benchmark. To enlarge the sample we built a generator that writes runs in the identical recorded format: the two topologies with their measured per-service baselines plus random topologies, nine fault classes that propagate up and down the graph, overlapping faults and unlabelled noise (noisy neighbours, deploys, missing readings). Settings were chosen on a 16-campaign tuning split (170 faults, seed 1) and every reported synthetic number is from a disjoint 16-campaign split (178 faults, seed 2) scored once. The generator also takes an amplification factor A that scales how strongly

callers feel a fault (retry storms) and a class filter; it underlies the amplification study.

Metrics. A fault is *detected* if an alarm during it (or within a 60 s grace) places a truly involved service in the top three; it is *caught and named* (C+N) if, in addition, the faulted service is first at the last alarming window before the fault heals. A@k is the share of detected faults with the true cause in the top k. False alarms are alarm episodes outside every fault window per fault-free hour; alarm time is the fraction of fault-free time in alarm. Detection delay is measured from fault onset. Service-level detection is scored with ROC and precision–recall curves on examples that are the faulted service during a fault (positive) or any service when no fault is active (negative), excluding examples with a container fact, which are identical for every detector.

Statistics. Intervals are 95% percentile bootstrap over faults (Efron [47]); comparisons between engines are *paired* bootstrap differences over the same faults, with two-sided p values; seed-averaged results are means over seeds. Overlapping faults within a run are not independent, so intervals are, if anything, optimistic; with 62 real faults a difference below about 8 points cannot be resolved, and we make no claim below that. All results are reproducible from the project's scripts (`ml:compare`, `ml:hybrid`, `ml:predict`, `paper/exp`).

A measurement bug. Each recorded run contains two fault campaigns that both number their faults from one, and the ledger reader kept only the second, hiding 14 of 31 faults per system for every engine and counting their outages as false alarms. We fixed the reader, added a regression test, and every number here uses all 62 faults.

X. Results

A. Detection and root cause

Table I compares the engines in one harness. On the real systems the hybrid names the true cause first for 74% of faults against 65% for the rule layer, 71% for ranking by anomaly alone and 73% for the ML engine. The paired differences are in Fig. 5 and Table III: hybrid over rules is significant on the real systems (+9.7 points, 95% CI [+1.6, +17.7], p=0.032) but not on the synthetic set (+2.2, [−3.9, +8.4]); hybrid over the severity ranking is positive but not significant in either set (+3.2 and +1.1 points). The operating curves of Fig. 3 show why the comparison is delicate: the engines trade detection for false alarms along nearly overlapping curves, and the hybrid sits on the upper envelope in both sets, so its advantage is robustly about operating point rather than a single threshold. Fig. 8 breaks C+N down by fault class: the gain over the rules is concentrated in *stop* faults (16/16 against 9/16 on the real systems), which is the effect of the `docker stop` fact, and, on the synthetic set, *flap* faults (12/12 against 2/12).

B. Which detector

Fig. 4 and Table II score each detector as a service-level classifier. The result is not what a learned-detector paper would hope for: the plain rule z-score is as good as or better than every learned detector on discrimination (AUROC 0.696 against 0.635 for the Isolation Forest on the real systems, 0.873 against 0.862 on the synthetic set), and Mahalanobis is the best learned detector on the synthetic set (0.868). Separation on the real systems is weak for every detector (AUROC 0.62 to 0.70), because many real faults hit services that are nearly idle. The Isolation Forest is therefore kept for what it adds to the hybrid, not for discrimination alone: it is the quietest detector (alarm time of 5% against 9% for the rules on the real systems, Table I), it separates own from symptom evidence, and its alarms complement those of the rules, so their union detects more faults than either (92% against 89% on the synthetic set). The ensemble of four detectors was not run on the synthetic set to completion and is reported for the real systems only.

C. Detection delay and root-cause depth

Fig. 6 shows detection delay as a cumulative distribution. On the real systems the change-point alarm lifts the detected fraction from 90% to 95% and moves the median from 10.5 s to 4.9 s; on the synthetic set, whose faults are abrupt, the curves almost coincide. It does so at a price visible in Table I: alarm time rises from 7% to 16% on the real systems. Fig. 7 shows the true cause in the top k: the ML-based rankers reach it first more often than the rules on the real systems and the curves converge by k=2 to 3 near the 90% detection ceiling, so the remaining misses are mostly faults the engine does not see (brownouts on near-idle services) rather than faults it ranks badly.

D. Ablation

Fig. 9 removes one mechanism at a time. The single most important component is the `docker stop` fact: without it C+N falls by 19 points on the real systems and 16 on the synthetic set, because every stopped service goes unnamed. The crash-loop fact (3 and 7 points), the range guard (4 and 11 points) and the random walk (3 and 1 points) each contribute. Two mechanisms act on the contract rather than on accuracy: removing the one-sided features raises alarm time on the real systems from 5% to 30%, and alarming at level one instead of two raises it to 22% and false alarms to 11.3 per hour while changing C+N by only +2 points. The one-sided features do not matter on the synthetic set, where restarts do not lower memory: an effect of the real systems that the simulator does not reproduce.

E. When does the graph matter? The amplification study

Standard benchmarks have faults whose own service is also the loudest, which is the case where ranking by severity cannot fail. To probe the regime in which it can, we regenerate soft faults (brownout, latency, memory) with the amplification factor $A \in \{1, 1.5, 2, 3, 4\}$, keep the hybrid's alarms identical for every ranker and vary only the ranking (Fig. 10). As A grows, the share of faults in which some victim is louder than the true cause rises from 15% to 25% (panel c), the severity ranking falls from 67% to 62%, and the walk holds near 67%, so the gain of the walk over severity grows from about one point to about five (panel b); but its intervals include zero at every level ($+4.7$ points, 95% CI $[-0.9, +11.3]$, at A=4). The

TABLE I
DETECTION AND ROOT CAUSE IN ONE HARNESS (POINT ESTIMATES WITH 95% BOOTSTRAP INTERVALS OVER FAULTS). DET.: FAULTS DETECTED (%). A@1, A@3: TRUE CAUSE FIRST OR IN THE TOP THREE AMONG DETECTED (%). C+N: CAUGHT AND NAMED FIRST OVER ALL FAULTS (%). FA/H: FALSE-ALARM EPISODES PER FAULT-FREE HOUR. ALARM: SHARE OF FAULT-FREE TIME IN ALARM (%).

Engine	Det. [95% CI]	A@1	A@3	C+N [95% CI]	FA/h	Alarm
Real benches (62 faults, 217 min)						
Rules	90 [82, 97]	71	91	65 [53, 76]	9.2	9
Severity ranking	90 [82, 97]	79	88	71 [60, 82]	6.2	7
ML engine	87 [77, 95]	83	91	73 [61, 82]	2.3	5
Hybrid	90 [82, 97]	82	89	74 [63, 84]	6.2	7
Hybrid + change-point	95 [89, 100]	78	88	74 [63, 84]	6.2	16
Hybrid + learned dir.	90 [82, 97]	79	89	71 [60, 82]	6.2	7
Synthetic held-out (178 faults, 16 topologies)						
Rules	89 [84, 93]	78	95	70 [62, 76]	2.5	2
Severity ranking	92 [87, 96]	77	91	71 [64, 78]	0.8	6
ML engine	89 [85, 94]	78	93	70 [63, 76]	0.0	5
Hybrid	92 [87, 96]	79	92	72 [65, 79]	0.8	6
Hybrid + change-point	93 [89, 97]	77	91	72 [65, 79]	0.6	7
Hybrid + learned dir.	92 [87, 96]	79	93	72 [65, 79]	0.8	6

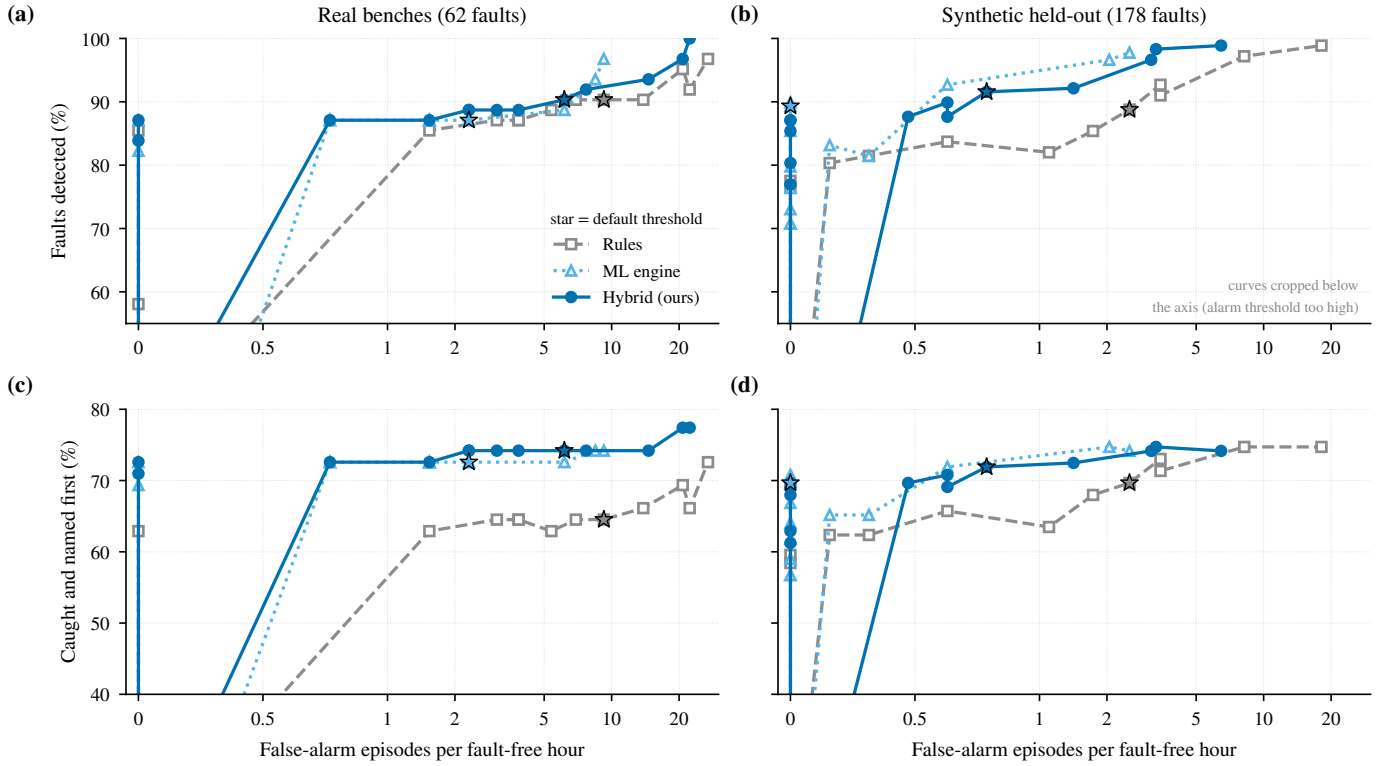


Fig. 3. Operating curves. Sweeping the alarm threshold of each engine traces detection rate (top) and caught-and-named rate (bottom) against false-alarm episodes per fault-free hour (symmetric-log axis); stars mark the default threshold. The hybrid sits on the upper envelope in both the real systems (a, c) and the synthetic held-out set (b, d).

strongest ranker on soft faults throughout is the own-versus-explained decomposition used by the rule layer (69 to 75%; $+5.7$ to $+9.4$ points over severity, with intervals excluding zero at $A{=}1.5$ and $A{=}4$). A router that uses it whenever no container fact holds did not change the full benchmarks (within ± 2 points), so the shipped ranking is unchanged. We read this as: graph-aware ranking is a modest, regime-dependent gain, and the decomposition matters more than the walk.

F. The cascade model

Fig. 11 evaluates who-is-next predictions at the first alarm of each fault: for every service still healthy, the probability that it turns unhealthy before the fault ends. On the synthetic set (159 faults, 1 196 predictions, 80 cascades) the learned propagation model reaches AUROC 0.847 against 0.784 for the same noisy-OR with every edge fixed at 0.7, a Brier score of 0.055 against 0.060 and an ECE of 0.039 against 0.058. The reliability curve is noisy in the upper bins, which hold few predictions. *On the real systems the experiment cannot be done well:* only 5 of 420 predictions were followed by a

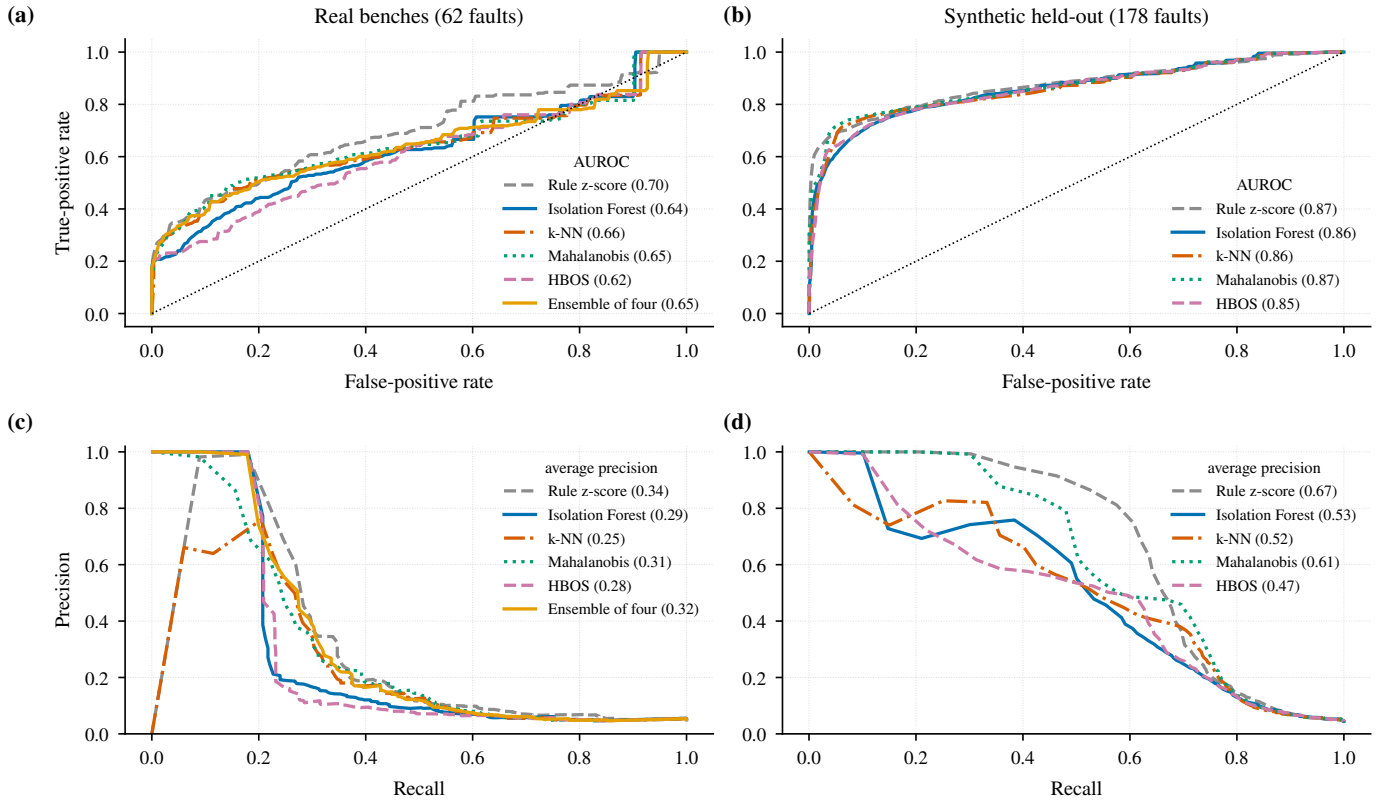


Fig. 4. Service-level detection by detector: ROC (top) and precision–recall (bottom) for the real systems (left) and the synthetic held-out set (right); legends give AUROC and average precision (means over three seeds for the randomised detectors). Positives are the faulted service during a fault; negatives are any service when no fault is active; container facts are excluded.

TABLE II
SERVICE-LEVEL DETECTION BY DETECTOR (MEAN ± S.D. OVER THREE SEEDS; THE RULE LAYER IS DETERMINISTIC). DASHES: NOT RUN.

	Real		Synthetic	
Detector	AUROC	AP	AUROC	AP
Rule z-score	0.696±0.000	0.340±0.000	0.873±0.000	0.666±0.000
Isolation Forest	0.635±0.009	0.285±0.003	0.862±0.004	0.528±0.021
k-NN	0.657±0.010	0.254±0.024	0.860±0.001	0.517±0.004
Mahalanobis	0.648±0.003	0.309±0.002	0.868±0.001	0.608±0.006
HBOS	0.619±0.008	0.279±0.004	0.853±0.004	0.473±0.030
Ensemble of four	0.648±0.001	0.323±0.003	–	–

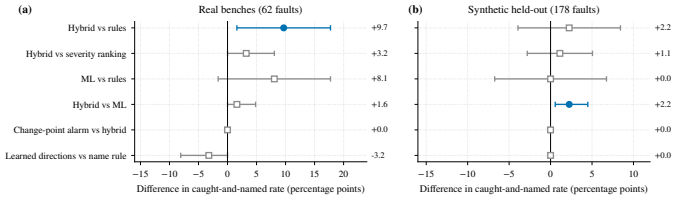


Fig. 5. Paired differences in caught-and-named rate between engines (percentage points, 95% paired-bootstrap interval); filled markers exclude zero.

cascade (54 faults), so the real-system comparison (AUROC 0.685 against 0.661; the live mapper's own risk values 0.530) is not informative. We state this because the real faults are mostly single-service outages that the systems absorb; whether learned propagation helps on a real cascade is untested.

TABLE III
PAIRED BOOTSTRAP TESTS BEHIND FIG. 5 (DIFFERENCE IN CAUGHT-AND-NAMED RATE).

Set	Contrast	Δ (pp)	95% CI	p
Real	Hybrid vs rules	+9.7	[+1.6, +17.7]	0.032
Real	Hybrid vs severity ranking	+3.2	[+0.0, +8.1]	0.256
Real	ML vs rules	+8.1	[-1.6, +17.7]	0.120
Real	Hybrid vs ML	+1.6	[+0.0, +4.8]	0.707
Real	Change-point alarm vs hybrid	+0.0	[+0.0, +0.0]	1.000
Real	Learned directions vs name rule	-3.2	[-8.1, +0.0]	0.257
Synth.	Hybrid vs rules	+2.2	[-3.9, +8.4]	0.516
Synth.	Hybrid vs severity ranking	+1.1	[-2.8, +5.1]	0.661
Synth.	ML vs rules	+0.0	[-6.7, +6.7]	1.000
Synth.	Hybrid vs ML	+2.2	[+0.6, +4.5]	0.036
Synth.	Change-point alarm vs hybrid	+0.0	[+0.0, +0.0]	1.000
Synth.	Learned directions vs name rule	+0.0	[+0.0, +0.0]	1.000

G. Learned directions

Fig. 12 reports direction accuracy on observed pairs that are true calls. With names available the name rule is correct for 21 of 31 real pairs (and leaves 10 undecided), the learned rule for 28 of 31 (3 wrong). With service names hidden the name rule is undecided for every pair while the learned rule is unchanged, and the same holds on the synthetic set (95 of 204 against 197 of 204 correct). The effect on ranking is smaller than the effect on directions: with names available the learned directions do *not* beat the name rule (71% against 74% on the real systems, −3.2 points, not significant), and with names

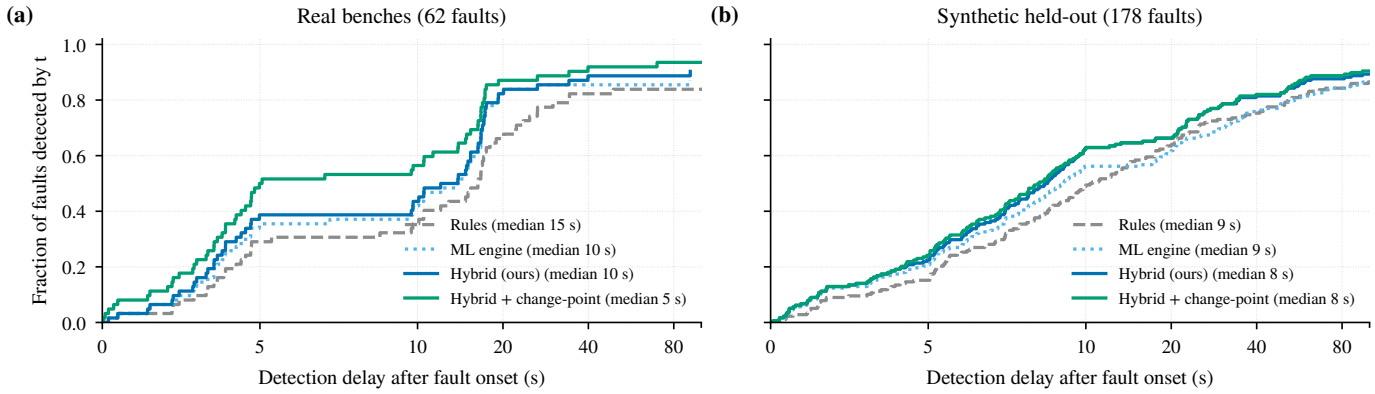


Fig. 6. Detection delay after fault onset as a cumulative distribution (symmetric-log axis); legends give the median over detected faults. The curves plateau at each engine's detection rate.

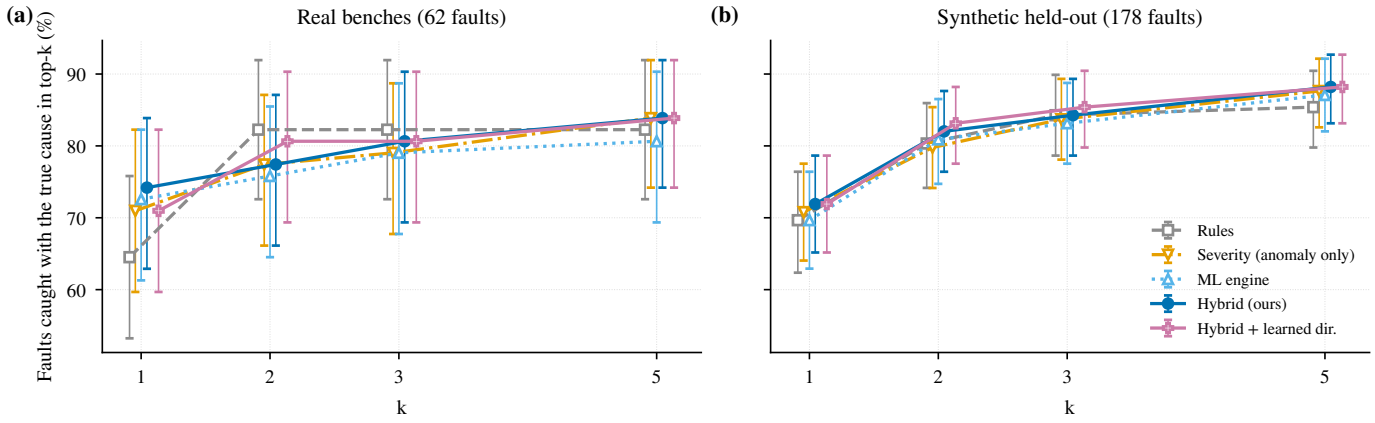


Fig. 7. Share of faults caught with the true cause in the top k, with 95% bootstrap intervals (offset for legibility), for the real systems (a) and the synthetic held-out set (b).

hidden they reach 71% against 69% for the name rule and 73% for the true graph. So learned directions are a robustness mechanism, not an accuracy gain.

H. Early warning and fault diagnosis

The trend forecast warns before the alarm for 16 of 178 synthetic faults (7 of 18 memory leaks) with a median lead of 30 s and about one false warning per fault-free hour (Fig. 13); on the real systems it fired for one of 62 faults, since those faults are abrupt. Fig. 14 shows fault-type diagnosis: with the true service the forest is correct for 99% of synthetic faults 60 s after the alarm; on the engine's own suspect it is 77%, because a wrong suspect is rarely typed correctly. On the real systems a hand-written decision list generalises best (59% on the suspect, 84% on the true service), because real flaps are pause cycles and real brownouts on idle services show no throttling: differences between the simulator and the real systems that the learned models absorb as synthetic-specific patterns.

I. Cost

The pipeline is unsupervised, needs no GPU and runs on one core. In a single replay of the two real systems (12 and 13 services) one decision window, covering the per-service forests, facts, random walk and cascade risk, took about 6 to 7 ms on average, far below the 5 s collection period. We did *not* measure how the cost grows with the number of services beyond these sizes, so we make no scaling claim; the per-service forests are independent and the walk is linear in edges times iterations, but this was not benchmarked.

J. Case studies

Fig. 15 shows three real incidents, chosen by class before looking at the outcome. In (a) a memory leak overlaps several outages: the engine names the stopped containers first, as it should, and only a model of concurrent faults could do better. In (b) a crash loop is named correctly about 30 s after its first kill and stays named between kills. In (c) a brownout on an almost idle service never raises an alarm, which is the open problem of Fig. 8.

XI. Discussion and Threats to Validity

What the evidence supports. Three results are robust: the fact-aware hybrid names the true cause more often than the rules on the real systems; the container facts are the single most valuable component; and the learned propagation model discriminates better than fixed edge probabilities on observed

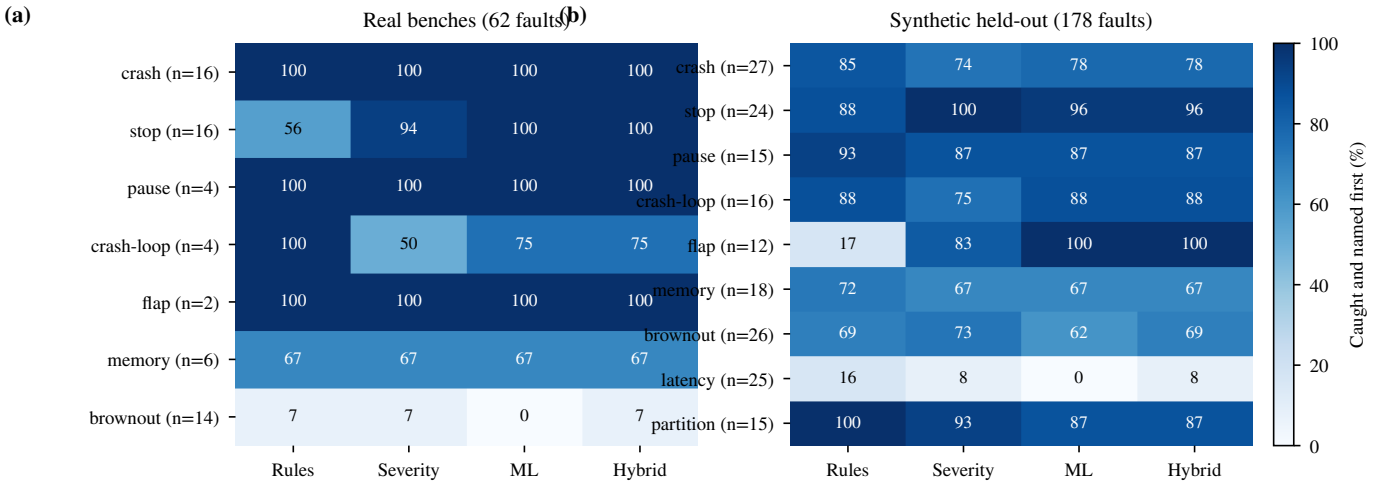


Fig. 8. Caught-and-named rate by fault class (percent; n faults per class), for four rankers on the real systems (a) and the synthetic held-out set (b). Brownouts are the open problem on the real systems.

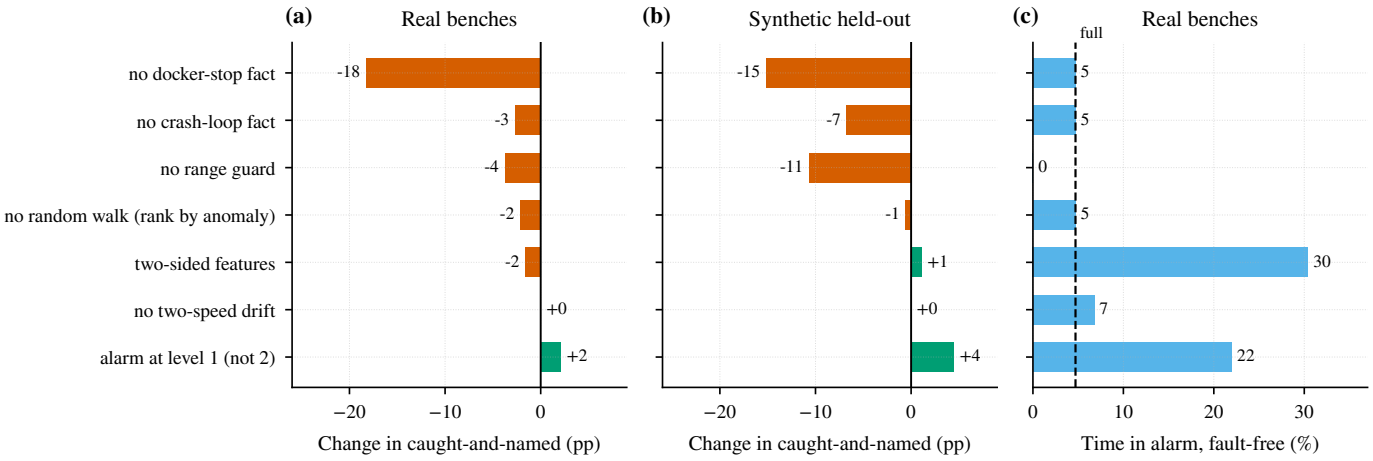


Fig. 9. Ablation. Each bar removes one mechanism from the full engine. (a), (b): change in caught-and-named rate (percentage points) on the real systems and the synthetic held-out set. (c): share of fault-free time in alarm on the real systems; the dashed line marks the full engine.

cascades. Three claims we expected to hold did not: the walk is not statistically better than ranking by anomaly on these benchmarks, learned directions do not beat the name rule when names exist, and onset-precedence re-ranking was neutral. We report them because a pipeline that is honest about where it stops is more useful to an operator than one that is not.

Construct validity. Ground truth is the injector's ledger. It records the structural root cause and the services able to be affected; whether an affected service visibly changes depends on protection the ledger does not model, so cascade recall is a lower bound on true propagation. The evidence weighting of Section IV is not isolated by the offline benchmarks because the recordings do not retain edge evidence. **Internal validity.** Settings were chosen by us. On the synthetic track they were chosen on a disjoint tuning split and scored once; on the real track they were not tuned on a held-out system, so the real numbers are optimistic by an unmeasured margin. The two container facts were found by inspecting the real runs. **External validity.** Two real applications and one capture; architecture independence is argued from the cross-system check and the synthetic topologies, not proven, and the simulator is cleaner than a real system (for example restarts do not lower memory in it). **Conclusion validity.** With 62 real faults a difference below about eight points cannot be resolved, and overlapping faults make the intervals optimistic.

Where a learned or language-model agent could go. The assistant here is deliberately a transparent composition of statistics. A language-model agent could sit on top of its output (cause, evidence, runbook) to answer follow-up questions or draft a change, with the structured evidence as its only grounding; evaluating that requires a user study and is outside this paper.

XII. Conclusion

We presented an instrumentation-free, agent-assisted diagnosis pipeline whose core is a probabilistic cascading-failure model with learned edge propagation, a hybrid detector in which container facts are evidence, and a ranker that separates a service's own deviation from what its neighbours explain. On 62 real faults the hybrid names the true cause first

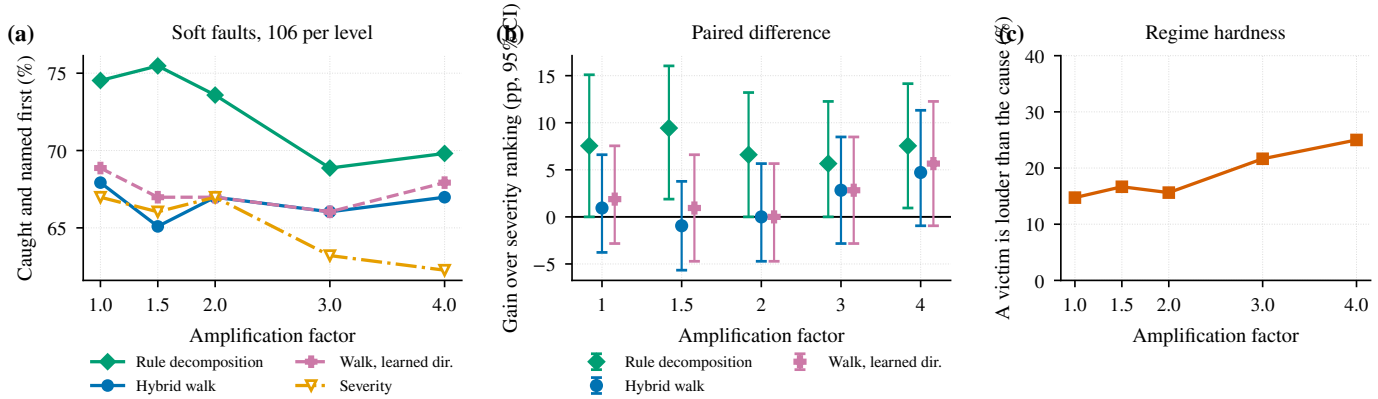


Fig. 10. Amplification study on soft faults (106 per level). (a) Caught-and-named rate of four rankers on identical alarms. (b) Gain over the severity ranking with 95% paired-bootstrap intervals. (c) Share of faults in which a victim is louder than the true cause.

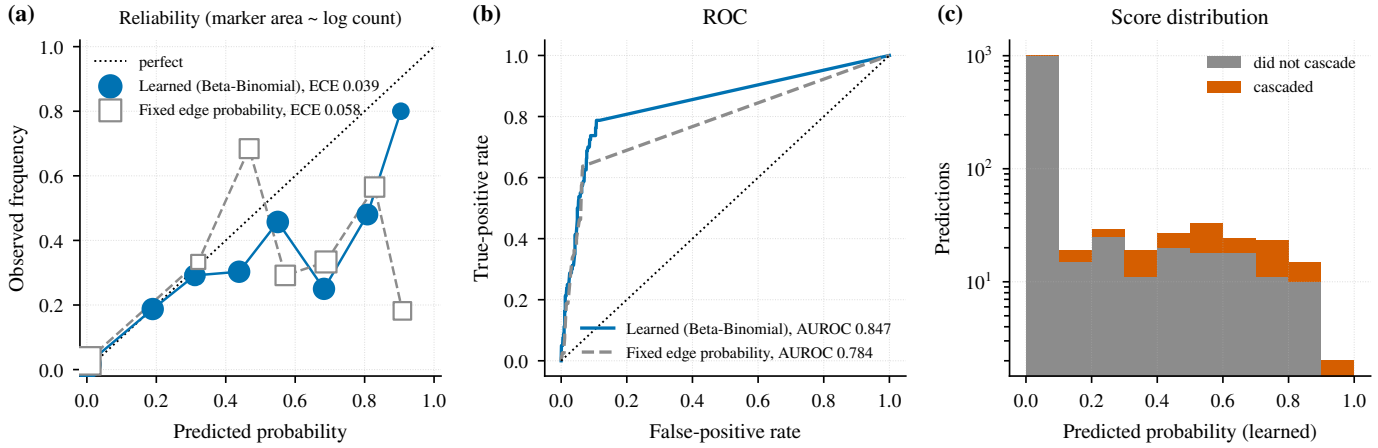


Fig. 11. Who-is-next prediction on the synthetic held-out set. (a) Reliability diagram with equal-width bins and marker area proportional to the logarithm of the bin count. (b) ROC curves. (c) Distribution of the learned predicted probabilities.

for 74% against 65% for the rules (paired $+9.7$ points, CI $[+1.6, +17.7]$); learned propagation discriminates better than fixed edge probabilities on 80 observed cascades (AUROC 0.847 against 0.784); learned directions stay correct when names are hidden; and change-point alarms halve the median detection delay on the real systems. We also showed what does not help and corrected a measurement bug that had hidden half of the real faults. The main open problems are brownouts on near-idle services, which no engine here catches, and validating learned propagation on real multi-service cascades, which the available recordings do not contain. Future work is a larger measured campaign on more systems, a public labelled dataset for comparability, promotion of learned directions and change-point alarms into the live system behind its operator switch, and a user study of an assistant layer built on the structured evidence.

References

[1] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*. Springer, 2017, pp. 195–216.

[2] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

[3] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," *ACM Trans. Intelligent Systems and Technology*, vol. 12, no. 6, pp. 81:1–81:45, 2021.

[4] S. Zhang, P. Jin, Z. Lin, Y. Sun, B. Zhang, S. Xia, Z. Li, Z. Zhong, M. Ma, W. Jin, D. Zhang, Z. Zhu, and D. Pei, "Robust failure diagnosis of microservice system through multimodal data," *IEEE Trans. Services Computing*, 2023.

[5] Y. Sun, Z. Lin, B. Shi, S. Zhang, S. Ma, P. Jin, Z. Zhong, L. Pan, Y. Guo, and D. Pei, "Interpretable failure localization for microservice systems based on graph autoencoder," *ACM Trans. Software Engineering and Methodology*, vol. 34, no. 2, 2024.

[6] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Eadro: An end-to-end troubleshooting framework for microservices on multi-source data," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2023, pp. 1750–1762.

[7] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[8] ——, "Isolation-based anomaly detection," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 1, pp. 3:1–3:39, 2012.

[9] S. Ramaswamy, R. Rastogi, and K. Shim, "Efficient algorithms for mining outliers from large data sets," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2000, pp. 427–438.

[10] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2000, pp. 93–104.

[11] M. Goldstein and A. Dengel, "Histogram-based outlier score (HBOS): A fast unsupervised anomaly detection algorithm," in *KI-2012: Poster and Demo Track*, 2012, pp. 59–63.

[12] H. Hotelling, "The generalization of Student's ratio," *Annals of Mathematical Statistics*, vol. 2, no. 3, pp. 360–378, 1931.

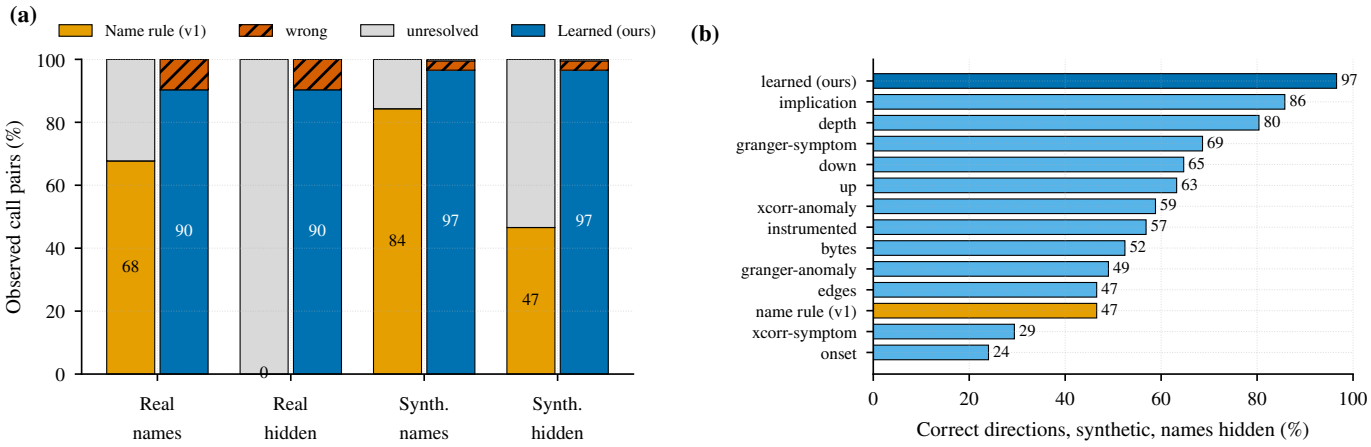


Fig. 12. Call directions. (a) Share of observed true-call pairs oriented correctly, wrongly, or left unresolved by the name rule and by the learned rule, with service names shown or hidden. (b) Correct share of each individual signal on the synthetic set with names hidden.

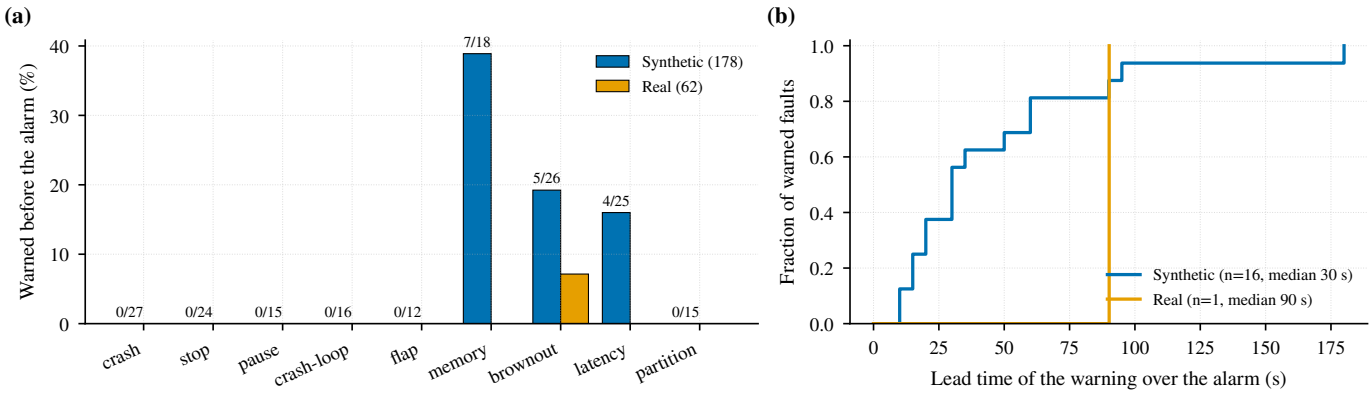


Fig. 13. Early warning. (a) Share of faults warned before the alarm, by fault class. (b) Lead time of the warnings over the alarm as a cumulative distribution.

[13] P. C. Mahalanobis, "On the generalised distance in statistics," *Proc. National Institute of Sciences of India*, vol. 2, no. 1, pp. 49–55, 1936.

[14] O. Ledoit and M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365–411, 2004.

[15] F. R. Hampel, "The influence curve and its role in robust estimation," *Journal of the American Statistical Association*, vol. 69, no. 346, pp. 383–393, 1974.

[16] P. J. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation," *Journal of the American Statistical Association*, vol. 88, no. 424, pp. 1273–1283, 1993.

[17] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[18] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 15:1–15:58, 2009.

[19] R. Wu and E. J. Keogh, "Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress," *IEEE Trans. Knowledge and Data Engineering*, vol. 35, no. 3, pp. 2421–2429, 2023.

[20] S. Kim, K. Choi, H.-S. Choi, B. Lee, and S. Yoon, "Towards a rigorous evaluation of time-series anomaly detection," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 36, no. 7, 2022, pp. 7194–7201.

[21] N. Tatbul, T. J. Lee, S. Zdonik, M. Alam, and J. Gottschlich, "Precision and recall for time series," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[22] M. Kim, R. Sumbaly, and S. Shah, "Root cause detection in a service-oriented architecture," in *Proc. ACM SIGMETRICS Int. Conf. Measurement and Modeling of Computer Systems*, 2013, pp. 93–104.

[23] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020, pp. 1–9.

[24] P. Wang, J. Xu, M. Ma, W. Lin, D. Pan, Y. Wang, and P. Chen, "CloudRanger: Root cause identification for cloud native systems," in *Proc. IEEE/ACM Int. Symp. Cluster, Cloud and Grid Computing (CCGRID)*, 2018, pp. 492–502.

[25] J. Lin, P. Chen, and Z. Zheng, "Microscope: Pinpoint performance issues with causal graphs in micro-service environments," in *Proc. Int. Conf. Service-Oriented Computing (ICSOC)*, 2018, pp. 3–20.

[26] M. Ma, J. Xu, Y. Wang, P. Chen, Z. Zhang, and P. Wang, "AutoMAP: Diagnose your microservice-based web applications automatically," in *Proc. The Web Conference (WWW)*, 2020, pp. 246–258.

[27] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, and X. Li, "MicroRank: End-to-end latency issue localization with extended spectrum analysis in microservice environments," in *Proc. The Web Conference (WWW)*, 2021, pp. 3087–3098.

[28] A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, and M. Kocaoglu, "Root cause analysis of failures in microservices through causal discovery," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[29] M. Li, Z. Li, K. Yin, X. Nie, W. Zhang, K. Sui, and D. Pei, "Causal inference-based root cause analysis for online service systems with intervention recognition," in *Proc. ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, 2022, pp. 3230–3240.

[30] R. Xin, P. Chen, and Z. Zhao, "CausalRCA: Causal inference based precise fine-grained root cause localization for microservice applications," *Journal of Systems and Software*, vol. 203, p. 111724, 2023.

[31] H. Wang, Z. Wu, H. Jiang, Y. Huang, J. Wang, S. Kopru, and T. Xie, "Groot: An event-graph-based approach for root cause analysis in industrial settings," in *Proc. IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2021, pp. 419–429.

[32] D. Kempe, J. Kleinberg, and É. Tardos, "Maximizing the spread of

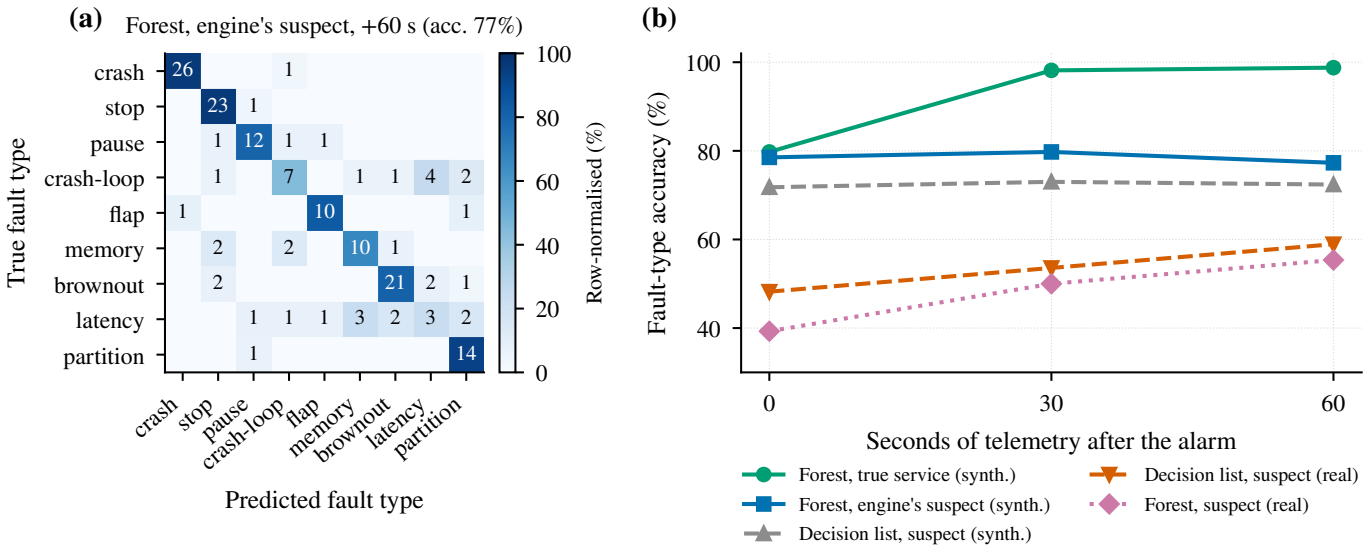


Fig. 14. Fault-type diagnosis. (a) Confusion matrix (counts; shading is row-normalised) of the random forest on the engine's suspect 60 s after the alarm, synthetic held-out set. (b) Accuracy against seconds of telemetry after the alarm, for the true service and the engine's suspect.

influence through a social network," in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2003, pp. 137–146.
[33] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
[34] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. Int. Conf. Machine Learning (ICML)*, 2005, pp. 625–632.
[35] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 70, 2017, pp. 1321–1330.
[36] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.
[37] R. P. Adams and D. J. C. MacKay, "Bayesian online changepoint detection," arXiv:0710.3742, 2007.
[38] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Tech. Rep., 1999.
[39] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009, pp. 758–759.
[40] C. W. J. Granger, "Investigating causal relations by econometric models and cross-spectral methods," *Econometrica*, vol. 37, no. 3, pp. 424–438, 1969.
[41] T. Schreiber, "Measuring information transfer," *Physical Review Letters*, vol. 85, no. 2, pp. 461–464, 2000.
[42] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
[43] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Wadsworth, 1984.
[44] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.
[45] Y. Gan, Y. Zhang, D. Cheng *et al.*, "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proc. Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 3–18.
[46] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding, "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Trans. Software Engineering*, vol. 47, no. 2, pp. 243–260, 2021.
[47] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.



Fig. 15. Three real incidents. Heat maps show the ML level of the most relevant services; the blue box is the fault, grey dashed boxes are other faults running at the same time, black squares mark container facts, the green line is the first alarm in the window (an alarm already ongoing from an earlier fault starts at the left edge), and the strip below each map shows whether the engine's top-ranked service was the true cause.